FleetX: A Modular-Monolith Intelligent Fleet and Logistics Management System with Real-Time Simulated Telemetry

Department of Artificial Intelligence & Data Science, Arya College of Engineering & IT, Jaipur, India
Project-Based Learning (PBL) Capstone | Academic Year 2026–27 | Guide: Er. Ram Babu Buri

Abstract*—Mid-sized logistics operators often keep fleet records, driver credentials, maintenance schedules and cargo manifests in disconnected spreadsheets and paper files, causing unplanned breakdowns, unsafe dispatch decisions and poor shipment visibility. Student systems addressing this frequently degrade into tangled monoliths or overreach into fragile microservices. This paper presents FleetX, an Intelligent Fleet and Logistics Management System developed as a Java Project-Based Learning capstone. FleetX adopts a Modular Monolith architecture: one deployable Spring Boot 3.5.x application on Java 21 whose isolated domain packages communicate only through immutable DTOs and explicit lookup interfaces, never through shared entities or repositories. The MySQL 8.4 LTS schema is governed exclusively by Flyway migrations (V100–V501), with Hibernate restricted to validation. Spring Security provides server-side sessions, CSRF protection, BCrypt hashing and role-based access control. Five modules deliver identity administration; vehicle lifecycle management with fuel-efficiency calculation and preventive-maintenance alerts; driver and trip operations with eligibility-checked dispatch and a trip state machine; shipment management with Haversine-based distance and ETA estimation; and a real-time control tower. The control tower streams simulated GPS telemetry over STOMP WebSockets to a React 18 and TypeScript dashboard combining a 2D Leaflet map with a 3D Three.js view, geofence breach alerting and CSV reporting. Telemetry is explicitly labelled as simulated, and routing uses great-circle estimation rather than proprietary APIs. Verification comprised 88 of 88 passing backend tests, 41 of 41 passing frontend tests, and 10 of 10 successful Flyway migrations with restart persistence confirmed on MySQL 8.4 LTS without schema drift. FleetX shows that disciplined module boundaries and schema governance give student teams enterprise-grade maintainability without distributed-system overhead.

***Index Terms*—Modular monolith, fleet management, logistics, Spring Boot, Flyway schema migration, Haversine formula, WebSocket/STOMP, geofencing.**

I. INTRODUCTION

Road freight and last-mile logistics depend on three assets that must be coordinated continuously: vehicles, drivers and cargo. In many mid-sized transport firms these assets are tracked in separate tools. Maintenance logs live in spreadsheets, driver licences in paper files and trip manifests in yet another system, so no single party can answer simple operational questions: is this vehicle due for service, is this driver legally allowed to drive today, and where is this consignment right now?

This fragmentation has concrete costs. Without automated mileage- and interval-based reminders, preventive maintenance is missed and assets depreciate faster or fail on the road. Dispatchers can unknowingly pair a driver whose licence has expired, or a vehicle that is already committed or inoperable. During transit, dispatchers and customers rely on reactive phone enquiries because there is no shared view of cargo movement.

A second, less visible problem is architectural. Capstone systems built to address such domains tend to fail in one of two ways. A single-layer monolith accumulates tangled database queries and cross-feature dependencies until it cannot be changed safely. Conversely, a premature split into microservices introduces network failure modes, distributed data consistency and deployment overhead that a student team cannot realistically operate [3]. FleetX is designed to avoid both failure modes.

A. Contributions

- A **Modular Monolith** in Java 21 / Spring Boot 3.5.x with five isolated domain packages and zero direct cross-domain coupling, enforced through shared immutable DTOs and five explicit contract interfaces.
- **Schema governance** through ten Flyway migrations (V100–V501) with Hibernate `ddl-auto` fixed to `validate`.
- A **dispatch eligibility engine** and two **guarded finite state machines** (trip and shipment) that reject unsafe or illegal operations at the service layer.
- A **simulated telemetry pipeline** with circular and polygonal geofencing, streamed over STOMP/WebSocket to a dual 2D (Leaflet) and 3D (Three.js) control tower.
- An **honest verification record**: 88/88 backend and 41/41 frontend tests, and explicit labelling of every simulated or estimated quantity.

B. Paper Organization

Section II states the problem and requirements. Section III presents the architecture and Section IV the technology stack. Section V describes database design. Section VI covers the five modules, Section VII the algorithms, Section VIII security and Section IX the telemetry pipeline. Section X summarizes implementation and deployment, Section XI reports verification, Section XII discusses architectural trade-offs, Section XIII lists limitations, and Section XIV concludes.

II. PROBLEM STATEMENT AND REQUIREMENTS

A. Problem Statement

The problem is to design and implement an integrated fleet and logistics platform that (a) unifies vehicle, driver, trip, shipment and tracking data; (b) automates preventive-maintenance alerting; (c) prevents unsafe dispatch decisions; (d) gives real-time visibility of cargo in transit; and (e) remains maintainable by a five-member student team. The specification follows the structure recommended by IEEE 830 and ISO/IEC/IEEE 29148 [1], [2].

B. Functional Requirements

TABLE I
SUMMARY OF FUNCTIONAL REQUIREMENTS

ID	Requirement	Module
FR-1	Authenticate users; provision accounts with one of five roles; record audit events.	Identity
FR-2	Register commercial vehicles (VIN, plate, capacity, odometer, status).	Fleet
FR-3	Log fuel and compute km/L efficiency automatically.	Fleet
FR-4	Schedule maintenance and raise distance/time warnings.	Fleet
FR-5	Register drivers; validate licence expiry; track availability.	Drivers
FR-6	Validate driver and vehicle eligibility before trip creation.	Drivers
FR-7	Enforce trip lifecycle transitions with history.	Drivers
FR-8	Register shipments with tracking numbers and ordered waypoints.	Shipments
FR-9	Estimate route distance and ETA; enforce shipment lifecycle.	Shipments
FR-10	Simulate GPS and broadcast telemetry in real time.	Tracking
FR-11	Detect circular/polygonal geofence breaches and publish alerts.	Tracking
FR-12	Present KPIs and export CSV reports.	Tracking

C. Non-Functional Requirements

- **Maintainability:** strict module boundaries; no shared entities or repositories.
- **Data integrity:** schema changes only through versioned migrations; no automatic schema mutation.
- **Security:** authenticated access by default, CSRF protection, salted password hashing, least-privilege roles.
- **Responsiveness:** telemetry broadcast to dashboards with a design target below 500 ms.
- **Scientific honesty:** simulated or estimated outputs are labelled as such in APIs and UI.

III. SYSTEM ARCHITECTURE

A. Modular Monolith Pattern

FleetX is packaged as one deployable Spring Boot artifact, but internally divided into modules with the discipline of separate services [3], [4]. This keeps the single-process simplicity of a monolith (one build, one transaction manager, one database) while retaining the clear ownership boundaries that make microservices attractive. Fig. 1 shows the four tiers: a React presentation tier, a Spring Security perimeter, the modular application tier and a MySQL persistence tier.

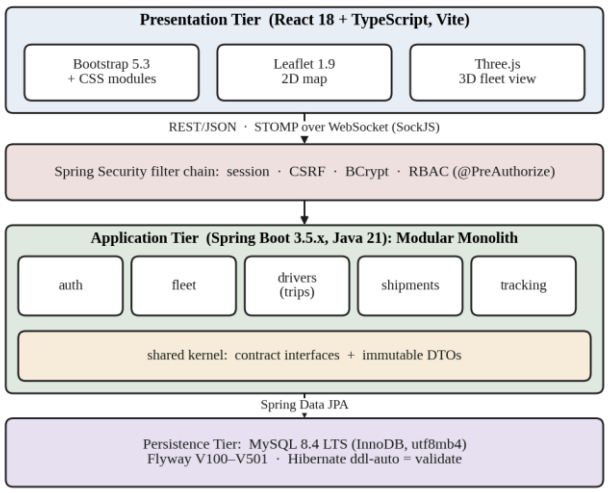


Fig. 1. Layered architecture of FleetX.

B. Domain Packages

The application tier contains five domain packages and a shared kernel under `com.fleetx`, summarized in Table II. Each domain package follows the same internal layering of controller, service, repository, entity and DTO.

TABLE II
DOMAIN PACKAGES AND RESPONSIBILITIES

Module	Package	Responsibility
Identity & Admin	`auth`	Users, roles, session login, administrator operations, audit log
Fleet & Assets	`fleet`	Vehicles, fuel records, maintenance records, service warnings
Driver & Trips	`drivers`	Drivers, licence checks, trips, trip history, performance metrics
Shipments	`shipments`	Shipments, waypoints, route plans, timeline events
Control Tower	`tracking`	Simulated GPS, telemetry, geofences, alerts, KPIs, CSV reports
Shared Kernel	`shared`	Contracts, immutable DTOs, security, WebSocket and error handling

C. Zero Direct Cross-Domain Coupling

The central design rule is that no domain package imports an entity or repository belonging to another domain. A static check of the source tree confirms that no `auth`, `fleet`, `drivers`, `shipments` or `tracking` class imports from any other domain package; every cross-domain import resolves into `shared`. Interaction instead happens through five contract interfaces and immutable summary DTOs (Fig. 2): `VehicleLookupService`, `DriverLookupService`, `TripLookupService`, `ShipmentLookupService` and `AlertPublisher`.

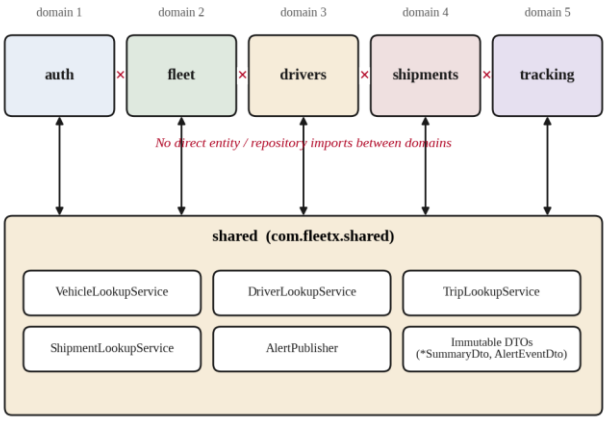


Fig. 2. Domains interact only through the shared contract layer.

The contracts are deliberately coarse and intention-revealing. For example, `isDriverEligibleForTrip(driverId)` hides the licence, availability and medical-certificate rules behind a single boolean, so the trip module never needs to know how eligibility is computed. Providers are registered as Spring beans, and a `@ConditionalOnMissingBean` fallback configuration supplies safe no-op implementations when a provider is absent. Team members could therefore build and test their own module before others were merged, without the fallbacks fabricating business data.

Cross-domain references are stored as plain identifiers. A trip, for instance, holds a vehicle ID and a driver ID rather than JPA relationships to foreign entities, so removing or replacing a module never forces a change in another module's persistence model.

D. Cross-Cutting Concerns

The shared kernel centralizes a uniform API envelope (`ApiResponse`, `PageResponse`), a global exception handler and a small hierarchy of domain exceptions that carry an HTTP status and a machine-readable code (for example `INVALID_STATE_TRANSITION`, `VEHICLE_NOT_AVAILABLE`, `DRIVER_NOT_ELIGIBLE`). Every module therefore reports errors in the same format. Controllers are additionally exposed under both `/api` and `/api/v1` prefixes.

IV. TECHNOLOGY STACK

TABLE III
TECHNOLOGY STACK

Layer	Technology
Backend	Java 21 LTS; Spring Boot 3.5.x (Spring MVC, Spring Data JPA, Spring Security, Spring WebSocket/STOMP)
Database	MySQL 8.4 LTS (InnoDB, utf8mb4), persistent containerized storage
Migration	Flyway, migrations V100–V501
Frontend	React 18, TypeScript, Vite, Bootstrap 5.3 with custom CSS modules
Visualization	Leaflet 1.9 + OpenStreetMap (2D); Three.js r160+ (3D)
Real-time	Spring STOMP over WebSocket, SockJS fallback
Testing	JUnit 5, Mockito, AssertJ, Spring Boot Test; Vitest, React Testing Library

Java 21 and Spring Boot were chosen for strong typing, mature dependency injection and first-class security and WebSocket support. MySQL 8.4 LTS provides transactional InnoDB storage. React with TypeScript gives a type-checked single-page client, and Vite provides fast builds.

V. DATABASE DESIGN AND SCHEMA GOVERNANCE

A. Versioned Migrations

The relational schema comprises 21 tables created exclusively by ten Flyway migrations numbered V100 to V501 [5]. Each migration is applied once, in order, and recorded in Flyway's history table. Hibernate's `ddl-auto` property is locked to `validate`, so at start-up Hibernate only checks that entity mappings agree with the real schema and refuses to boot otherwise. This removes an entire class of problems in which an ORM silently alters production tables, and it makes schema drift between team members' machines detectable immediately.

B. Table Ownership

Every table belongs to exactly one module (Fig. 3), mirroring the code-level boundaries. Only the owning module reads or writes its tables; other modules obtain data through the contracts of Section III-C.

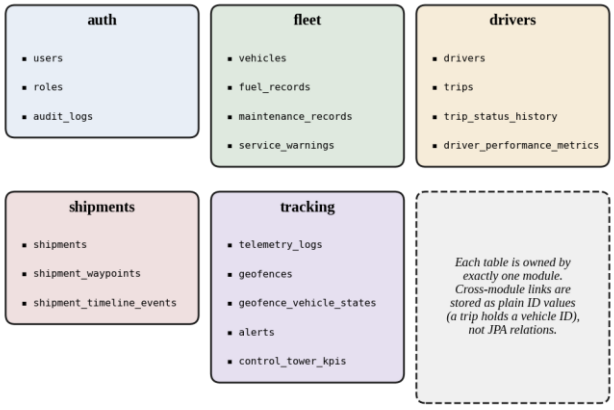


Fig. 3. Principal tables grouped by owning module.

Operational history is recorded in dedicated tables: `trip_status_history` stores every trip transition, `shipment_timeline_events` stores shipment milestones, `audit_logs` stores administrative actions, and `telemetry_logs` stores each simulated position fix together with a flag identifying it as simulated.

VI. FUNCTIONAL MODULES

A. Module 1: Identity and Administration

Five roles are supported: ADMIN, FLEET_MANAGER, DISPATCHER, DRIVER and VIEWER. Administrators provision users, update their details, enable or disable accounts and reset passwords. The authentication API exposes login, logout, current-user and CSRF-bootstrap endpoints. Significant actions are persisted by an audit service that writes in its own transaction (`REQUIRES_NEW`), so that an audit record is kept even when the surrounding business operation fails. Audit rows carry the acting user, action, entity, client address and a JSON details field, and are exposed to administrators as a filterable, paged list.

B. Module 2: Fleet and Asset Lifecycle

The vehicle registry stores identification (VIN, licence plate), cargo capacity, odometer and lifecycle status. Fuel

records capture litres, price or total cost, and odometer, from which efficiency is derived (Section VII-C). Maintenance records store the date and odometer of each service together with the next service-due distance and date. After each change the service re-evaluates warnings against the vehicle's current odometer and the date (Section VII-D).

C. Module 3: Driver and Trip Operations

The driver registry records licence validity and a medical certificate expiry. Availability is tracked through AVAILABLE, ON_DUTY, SUSPENDED and ON_LEAVE states. The central operation is trip creation, whose validation pipeline is shown in Fig. 4. A trip is created only if the vehicle is available and the driver is eligible: the driver must be active and available, with a licence that has not expired and a medical certificate, where recorded, that is still valid. On success the trip is persisted as ASSIGNED, the driver becomes ON_DUTY, the vehicle becomes IN_TRIP, an entry is appended to the trip history and a TRIP_ASSIGNED alert is published. Each of these cross-module side effects goes through a contract interface rather than a direct repository call.

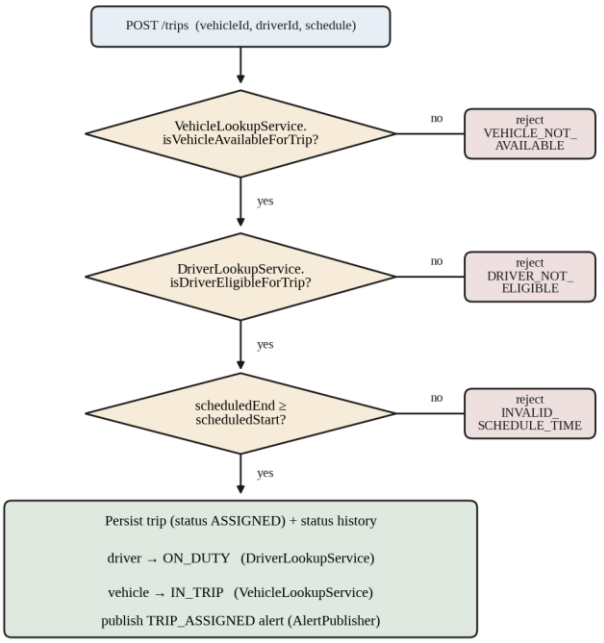


Fig. 4. Trip creation: eligibility validation and side effects.

The interaction among the trip service and the shared contracts is shown as a sequence in Fig. 5. Note that the trip service never touches a vehicle or driver table: steps 2, 3, 5 and 6 are all calls to lookup interfaces, and step 7 reaches the dashboards without the trip module knowing anything about WebSockets.

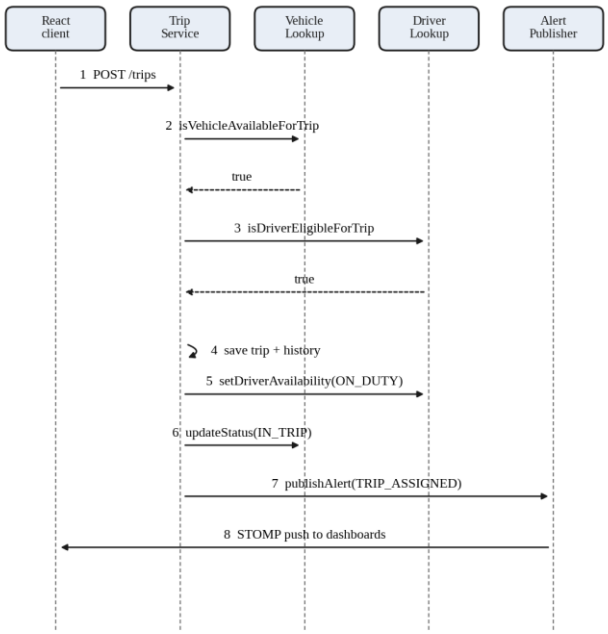


Fig. 5. Sequence of a trip dispatch through the shared contracts.

D. Module 4: Shipment Manifest and Route Logistics

Shipments carry a generated unique tracking number, sender and receiver details and parcel weight. Ordered waypoints define the planned route, and the route service computes total distance and ETA from them (Section VII-A). A shipment can be assigned to a trip, advanced through its lifecycle, marked delivered with a delivery confirmation, and queried by ID or tracking number. A timeline of events records each change, and a check operation flags shipments that have run late.

E. Module 5: Real-Time Telemetry and Control Tower

The tracking module runs the GPS simulator, stores telemetry, evaluates geofences, raises and acknowledges alerts, and produces KPI summaries and CSV exports. It is described in detail in Section IX.

VII. ALGORITHMS AND MATHEMATICAL FORMULATION

A. Haversine Distance and ETA

Straight-line distance between two coordinates is the great-circle distance on a sphere of mean radius R = 6371 km, computed with the Haversine formula [6]. With latitudes φ_1, φ_2 and coordinate differences $\Delta\varphi$, $\Delta\lambda$ in radians:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{1}$$

$$d = 2R \arcsin\left(\sqrt{a}\right) = 2R \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right) \tag{2}$$

The two forms of (2) are numerically equivalent; the implementation uses the `atan2` form, which is better conditioned for nearly antipodal points. For a route with ordered waypoints $P_1 \ldots P_n$ the total distance is

$$D = \sum_{i=1}^{n-1} d(P_i, P_{i+1}) \tag{3}$$

and the estimated duration, in minutes, uses a configurable average speed $\bar{v}$ and a per-stop allowance t_stop applied to the $n - 2$ intermediate waypoints:

$$T_{ETA} = \frac{60\,D}{\bar{v}} + \max(0,\, n-2)\cdot t_{stop} \tag{4}$$

Both quantities are explicitly labelled in the API as estimates: the distance is a straight-line figure, not road distance, and the ETA assumes constant speed and fixed stop allowance, not live traffic. No proprietary routing service is used.

B. Simulated GPS Interpolation

Between consecutive waypoints P_k and P_{k+1} the simulator advances the vehicle by a fraction f of the segment, clamped to [0, 1]:

$$P(t) = P_k + f(P_{k+1} - P_k), \quad f \in [0,1] \tag{5}$$

Heading is obtained from the bearing between the two points, and speed from the distance moved over elapsed time, capped at a maximum permitted value v_max (a validation limit of 300 km/h):

$$v = \min\left(v_{max},\, \frac{3600\,d}{\Delta t}\right) \tag{6}$$

C. Fuel Efficiency

For each fuel record the efficiency is the distance travelled since the previous reading divided by the fuel volume:

$$E = \frac{\Delta O}{V} = \frac{O_{curr} - O_{prev}}{V} \quad \text{[km/L]} \tag{7}$$

where O_prev is the odometer of the vehicle's latest fuel record or, for the first fill-up, its baseline odometer. If $\Delta O \leq 0$ (duplicate reading, top-up without driving, odometer reset) efficiency is recorded as null rather than a negative or distorted value, and V must be strictly positive.

D. Preventive-Maintenance Warnings

On each evaluation the latest scheduled service is compared with the current odometer and date. If the odometer has reached the due distance, or today is past the due date, a CRITICAL overdue warning is raised. Otherwise, if the vehicle is within 500 km or within 14 days of the due point, a WARNING of type service-due is raised. Each warning is stored and also published as a real-time alert.

E. Geofence Breach Detection

A circular geofence with centre c and radius r (metres) contains a point p when

$$\text{inside}_{circle}(p) \Leftrightarrow d(p,c) \leq r \tag{8}$$

Polygonal geofences use the ray-casting point-in-polygon test [7]: a horizontal ray from the point is intersected with each polygon edge and the parity of the crossing count decides containment (Algorithm 1).

```
Algorithm 1: isInsidePolygon(p, V[0..n-1])
 1: if n < 3 then return false
 2: inside <- false;  j <- n-1
 3: for i <- 0 to n-1 do
 4:   if p equals V[i] then return true
 5:   if (V[i].lat > p.lat) != (V[j].lat > p.lat)
 6:      and p.lon < x-intercept of edge (V[j],V[i])
 7:        inside <- not inside
 8:   j <- i
 9: return inside
```

The module stores, per geofence and vehicle, whether the vehicle was previously inside. A change from outside to inside, or inside to outside, produces exactly one enter or exit alert, so a vehicle that dwells inside a zone does not flood the dashboard.

F. Finite State Machines

Trip and shipment lifecycles are enforced in the service layer by explicit transition rules (Figs. 6 and 7). Any illegal request is rejected with the `INVALID_STATE_TRANSITION` error and terminal states cannot be left. Business-level names (DRAFT, IN_TRANSIT, ORDERED, DISPATCHED, RETURNED) are accepted as aliases of the internal enumeration values shown in brackets.

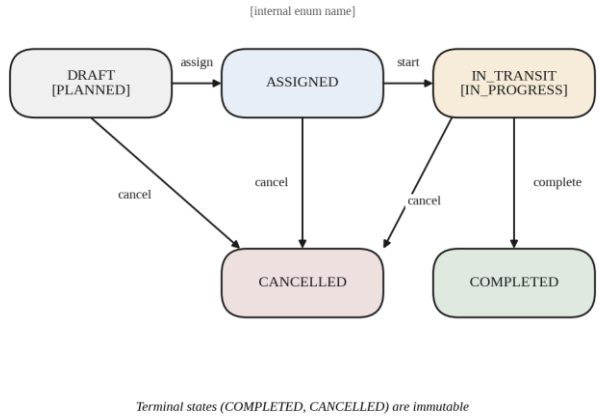


Fig. 6. Trip lifecycle state machine.

TABLE IV
TRIP TRANSITIONS AND GUARDS

From	To	Guard / effect
(create)	ASSIGNED	Vehicle available, driver eligible, valid schedule; driver ON_DUTY, vehicle IN_TRIP
DRAFT	ASSIGNED, CANCELLED	Permitted forward moves
ASSIGNED	IN_TRANSIT, CANCELLED	Start records actual start time; trip-started alert
IN_TRANSIT	COMPLETED, CANCELLED	COMPLETED reachable only from IN_TRANSIT
COMPLETED, CANCELLED	none	Terminal; no further change

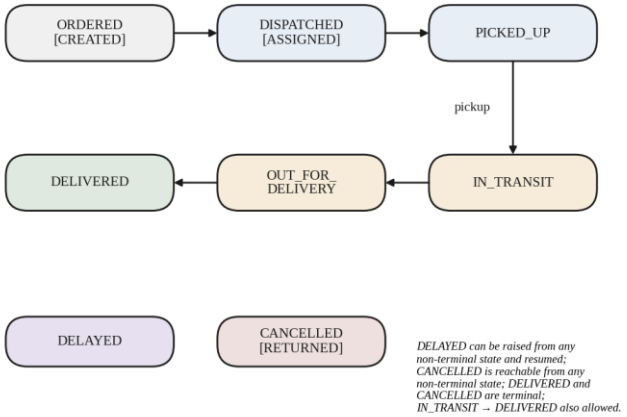


Fig. 7. Shipment lifecycle state machine.

TABLE V
SHIPMENT TRANSITIONS

From	To	Notes
ORDERED	DISPATCHED, PICKED_UP	Forward progression
DISPATCHED	PICKED_UP, IN_TRANSIT	Linked to a trip
PICKED_UP	IN_TRANSIT, OUT_FOR_DELIVERY	Pickup confirmed

From	To	Notes
IN_TRANSIT	OUT_FOR_DELIVERY, DELIVERED	Direct delivery allowed
Any non-terminal	CANCELLED, DELAYED	DELAYED may resume forward
DELIVERED, CANCELLED	none	Terminal

VIII. SECURITY DESIGN

FleetX uses stateful server-side sessions rather than bearer tokens. A successful login creates a session identified by the `JSESSIONID` cookie, configured HttpOnly with a Lax or Strict SameSite policy so that scripts cannot read it and cross-site requests do not carry it. State-changing requests must also present a CSRF token: a token repository issues an `XSRF-TOKEN` cookie that the single-page client reads and echoes in a request header. Only health, login, CSRF-bootstrap and the WebSocket handshake endpoints are exempt, and every other API route requires authentication by default. Passwords are hashed with BCrypt (cost factor 12) [8]. Authentication and authorization failures return structured JSON errors with 401 and 403 status codes.

Authorization is role-based and enforced twice: URL rules restrict `/admin` routes to the ADMIN role, and method-level `@PreAuthorize` annotations restrict each operation to the roles that need it (Table VI).

TABLE VI
ROLE-BASED ACCESS SUMMARY

Role	Principal privileges in the implementation
ADMIN	Everything, including user administration and audit log
FLEET_MANAGER	Vehicle and maintenance management; operational read/write across modules
DISPATCHER	Trips, drivers, shipments, simulator and geofence control, alerts
DRIVER	Fuel logging; start, complete or cancel own trips; delivery confirmation; telemetry ingest
VIEWER	Read-only reporting: summary and CSV export

IX. REAL-TIME TELEMETRY AND CONTROL TOWER

A. Simulation Engine

The GPS simulator is implemented inside the tracking service. A dispatcher starts a simulation for a vehicle or trip, supplying optional waypoints, a tick interval and a speed multiplier. The tick interval is bounded between 1 and 60 seconds and the multiplier between 0.5 and 20. Each running simulation is a task on a small scheduled thread pool, registered in a concurrent map so that starting a new run for the same vehicle cancels the previous task rather than leaking tasks. Simulations can be paused, resumed, stopped or advanced one step deterministically, which makes them testable.

B. Telemetry Pipeline

Each tick follows the pipeline of Fig. 8: the position is interpolated, validated, persisted as a telemetry record flagged as simulated, tested against active geofences and published. Alerts raised by any module (geofence enter or exit, overspeed, service-due, trip events) pass through the same `AlertPublisher` contract and reach the client on the same channel.

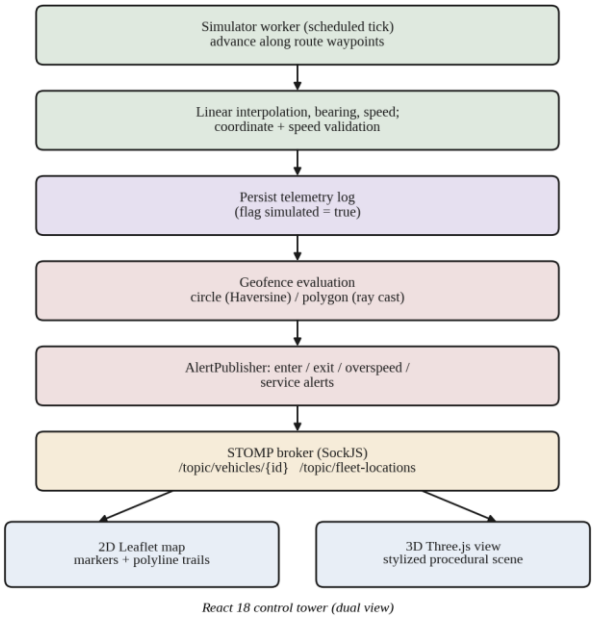


Fig. 8. Telemetry processing and broadcast pipeline.

Messages are published with Spring's messaging template to `/topic/vehicles/{id}` for a single vehicle and `/topic/fleet-locations` for the whole fleet, through an in-memory simple broker on the `/ws/fleetx` endpoint with SockJS fallback, following the STOMP protocol [9]. The design target is a broadcast latency below 500 ms.

C. Dual-View Control Tower

The React client subscribes to these topics and drives two synchronized views. The 2D view uses Leaflet 1.9 on OpenStreetMap tiles with a marker per vehicle and a polyline trail of recent positions. The 3D view is a stylized procedural Three.js scene whose objects are updated from the same telemetry stream. Both are labelled as simulated. The control tower also presents fleet KPIs and active alerts, and reports can be exported as CSV.

X. IMPLEMENTATION AND DEPLOYMENT

A. Code Organization

The backend consists of 147 Java source files and about 10,500 lines of code. Table VII shows how they are distributed. The size of the tracking package reflects its breadth: simulation, telemetry, geofencing, alerting, KPI aggregation and CSV reporting. Seeder classes in each module can populate demonstration data so that the system can be shown without manual data entry.

TABLE VII
BACKEND SOURCE METRICS

Package	Java files	Lines of code
auth	21	1,305
fleet	30	2,093
drivers (incl. trips)	25	1,812
shipments	23	1,729
tracking	23	2,870
shared	24	752
Application entry	1	14
Total	147	10,575

A static scan of import statements gives an architectural fitness measure: the number of imports from one domain package into another is zero for all five domains, while every domain imports from `shared`. This check is cheap to automate and could be added to the build as a guard against future erosion of the boundaries.

B. REST Interface

TABLE VIII
REST INTERFACE OVERVIEW

Module	Base path	Representative operations
Identity	`/api/v1/auth`	login, logout, current user, CSRF bootstrap
Admin	`/api/v1/admin`	user CRUD and status, audit-log listing
Fleet	`/api/v1/fleet`	vehicles, status, odometer, fuel and maintenance records, warnings
Drivers	`/api/v1/drivers`	registry, availability, performance
Trips	`/api/v1/trips`	create, status, start, complete, cancel, history
Shipments	`/api/v1/shipments`	create, assign trip, status, waypoints, deliver, timeline
Tracking	`/api/v1/tracking`	simulator control, telemetry, geofences, alerts, summary and CSV reports

C. Deployment View

At run time FleetX consists of three elements (Fig. 9): the browser-hosted React application, a single Spring Boot process and a MySQL 8.4 LTS database in a container with persistent storage. Flyway runs when the Spring Boot process starts, applying any pending migrations before the application serves requests. In development the CORS policy admits the Vite dev server origin with credentials, so that session and CSRF cookies work across the two ports.

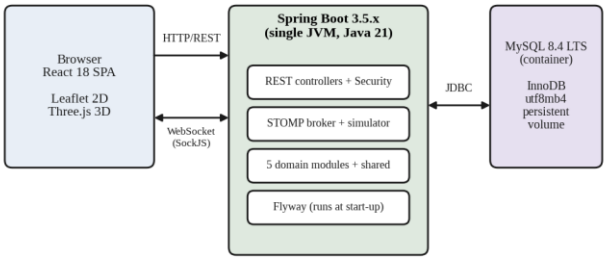


Fig. 9. Deployment view of FleetX.

XI. VERIFICATION AND RESULTS

A. Test Strategy

TABLE IX
TEST LAYERS

Layer	Tooling	Purpose
Unit	JUnit 5, Mockito, AssertJ	Business rules: state machines, eligibility, Haversine, fuel efficiency, geofence tests
Contract	JUnit 5, Spring Boot Test	Shared interfaces behave as documented
Schema	Flyway + Hibernate validate	Migrations and entity mappings agree
Integration / E2E	Spring Boot Test	Controllers, security and persistence working together
Frontend	Vitest, React Testing Library	Component and view behaviour

B. Results

Verification covered the backend, the frontend and the database layer. Backend tests use JUnit 5, Mockito, AssertJ and Spring Boot Test and include unit, contract, schema and end-to-end integration tests. Frontend tests use Vitest and React Testing Library.

TABLE X
VERIFICATION RESULTS

Check	Result
Backend automated tests	88 / 88 passing
Frontend tests (Vitest, RTL)	41 / 41 passing
Flyway migrations V100–V501	10 / 10 successful
Relational tables	21 (validated by Hibernate)
Restart persistence, MySQL 8.4 LTS	Verified; zero schema drift

The contract tests are especially relevant to the architecture: they check that each shared interface behaves as documented, which is what allows modules to evolve independently. Schema tests and `validate` mode together confirm that entity mappings and migrations agree. Restart persistence on MySQL 8.4 LTS confirmed that data survives container restarts and that no migration re-ran or altered existing tables.

XII. ARCHITECTURAL DISCUSSION

Table XI compares three structuring options for a system of this size. The modular monolith keeps the operational simplicity and single-transaction consistency of a monolith, and still enforces boundaries that make later extraction possible: because modules already talk only through contracts and plain IDs, any one of them could be moved behind a network interface by providing a remote implementation of its contract.

TABLE XI
QUALITATIVE COMPARISON OF STRUCTURING OPTIONS

Criterion	Classic monolith	Modular monolith	Microservices
Deployable units	1	1	Many
Module isolation	Convention only	Packages + contracts	Network boundary
Data consistency	Single transaction	Single transaction	Distributed
Operations effort	Low	Low	High
Fit for 5-person team	Risk of coupling	Good	Overhead risk

The approach has costs. Discipline relies on conventions and review rather than on a compiler or network, which is why an automated import check is attractive. Coarse contracts add some indirection, and a call through an interface is not as transparent as a direct repository query. For this project the team judged the indirection worthwhile for the independence it gave parallel development.

XIII. LIMITATIONS AND SCIENTIFIC DISCLOSURES

- **Simulated GPS.** Coordinates come from software interpolation along waypoints, not from devices. They are labelled “SIMULATED DATA” in APIs and UI.
- **Estimated distance and ETA.** The Haversine formula gives straight-line distance and ignores road geometry, one-way streets and traffic.
- **Stylized 3D view.** The Three.js scene is procedural and synchronized with telemetry; it is not a geographic reconstruction.

- **Single-node broker.** The in-memory STOMP broker and in-process simulator suit a single instance; horizontal scaling would need an external broker.
- **Scope.** The system is a capstone prototype and has not been load-tested against production fleet sizes.

Natural extensions are real GPS ingestion through the existing telemetry endpoint, a road-network routing engine for distance and ETA, an external message broker, and tamper-evident audit logging with hash chaining.

XIV. CONCLUSION

FleetX demonstrates that a student team can deliver an integrated fleet and logistics platform without choosing between an unstructured monolith and an over-distributed microservice design. Strict package isolation, contract interfaces with immutable DTOs, migration-governed schema evolution and guarded state machines kept the code maintainable and the operations safe. Real-time simulated telemetry, geofencing and a dual 2D/3D control tower provide the visibility that manual processes lack, and the transparent labelling of simulated and estimated data preserves scientific integrity. Passing 88 backend and 41 frontend tests and a clean ten-step migration history support the claim that the modular monolith is a practical architecture for capstone-scale systems.

REFERENCES

[1] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.

[2] Systems and software engineering — Life cycle processes — Requirements engineering, ISO/IEC/IEEE Std 29148:2018, 2018.

[3] S. Newman, *Building Microservices*. Sebastopol, CA, USA: O'Reilly Media, 2015.

[4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[5] Flyway Documentation, Redgate Software. [Online]. Available: https://flywaydb.org/documentation

[6] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[7] M. Shimrat, "Algorithm 112: Position of point relative to polygon," *Commun. ACM*, vol. 5, no. 8, p. 434, Aug. 1962.

[8] Spring Security Reference, VMware. [Online]. Available: https://docs.spring.io/spring-security/reference/

[9] STOMP Protocol Specification, Version 1.2. [Online]. Available: https://stomp.github.io/stomp-specification-1.2.html

PROJECT TEAM

MD ZIYAUL MUSTAFA [24EARAD100]

MD AMIR ALAM [24EARAD099]

KUSHARG GAUTAM [24EARAD087]

JITENDER KUMAR PASWAN [24EARAD071]

MAYANK KUMAR [24EARAD098]

Department of Artificial Intelligence & Data Science, Arya College of Engineering & IT, Jaipur

Guide: Er. Ram Babu Buri